SLE-4: Final Viva & Architecture Decision Record (ADR)

Course: 02AML204 – Introduction to Artificial Intelligence

PRN: 25UAM096

Name: Pranjal Vijay Mane

Division: B

Date: 11-10-2026

GitHub / Portfolio Link : https://github.com/manepranjal07-design/SLE-AI.git

---

PART A: Architecture Decision Record (ADR)

Title: Use A* Search with Manhattan Distance Heuristic instead of BFS as the main solver in the 8-Puzzle Solver System

Status: Accepted

Date: 11-10-2026

1. Context

The 8-Puzzle Solver System takes a 3x3 starting board and must return the shortest sequence of moves to the goal state (1, 2, 3, 4, 5, 6, 7, 8, 0). In SLE-2 I implemented two search strategies for this problem, uninformed BFS and informed A*, and profiled both on the same start state (1, 2, 3, 5, 0, 6, 4, 7, 8) using py-spy and a node counter. In SLE-3 I documented the architecture with the C4 model. The decision to be made was which algorithm should be the main solver in the final design, given that the solution must be optimal and the search should avoid exploring unnecessary states.

2. Decision

I decided to use A* search with the Manhattan distance heuristic as the main solver. BFS is kept only as a baseline for comparison, because A* returns the same optimal solution while expanding far fewer states.

3. Alternatives Considered

- **Alternative 1:** Breadth-First Search (BFS). It guarantees the shortest path, but it is uninformed and expands every state level by level, so it explores many irrelevant states and its frontier grows quickly.
- **Alternative 2:** Depth-First Search (DFS). It uses little memory, but it does not guarantee an optimal solution and can go deep into useless branches.
- **Alternative 3:** Greedy Best-First Search. It is fast, but it considers only the heuristic and ignores the cost so far, so the path it returns may not be the shortest.

4. Rationale / Justification

My SLE-2 profiling (5 runs per algorithm, averaged) showed that BFS expanded 33 nodes while A* expanded only 5 nodes on the same start state, which is about 6.6 times fewer expansions. The average running time was 0.044 ms for BFS and 0.028 ms for A*, and both algorithms returned the same optimal solution, so A* gave up nothing in solution quality. The py-spy flame graph also showed that BFS spent more time in neighbor generation and queue operations because of its larger frontier.

A* stays optimal because Manhattan distance is admissible: each move shifts one tile by one square, so the heuristic never overestimates the true number of moves left. The time

difference is only a few microseconds on this small instance, so I treat node expansions as the more reliable evidence, and it is the number that will grow with harder puzzles.

The decision also fits my SLE-3 design. The Heuristic Calculator is a separate container and the Search Engine uses a heapq priority queue for A* (a deque for BFS) inside the Frontier Manager component. The Memory State (Visited Set) tracks g-scores so states are not re-evaluated. Because search logic and heuristics are decoupled, a different heuristic can be swapped in without changing the traversal code.

5. Consequences

Positive:

- Far fewer nodes expanded than BFS (5 vs 33), while still returning the optimal path.
- The heuristic is modular, so it can be replaced or improved later without touching the core search code.

Negative / Trade-offs:

- A* is more complex to implement than BFS: it needs a priority queue, f(n) = g(n) + h(n), and a heuristic function.
- Each expansion has extra overhead from heap operations, and A* still stores many states in memory on harder instances, so it will not scale well to larger puzzles such as the 15-puzzle without IDA* or a stronger heuristic.
- My results come from one start state, so the comparison is limited and would be stronger with more test boards.

PART B: Short Summary of My SLE Journey

In SLE-1 I built a simple rule-based chatbot in Python (agent.py) and learned the basic perceive-process-act loop of an AI agent, while also practising how to document AI-assisted work through a contribution log and README. In SLE-2 I moved to search problems: I implemented BFS and A* with a Manhattan distance heuristic for the 8-Puzzle and profiled them with py-spy and a node counter over 5 runs each, finding that A* expanded 5 nodes against 33 for BFS and ran faster. In SLE-3 I modelled the 8-Puzzle Solver with the full C4 model (Context, Container, Component and Code levels) on draw.io, separating the input, search engine, heuristic, visited set and output modules. For SLE-4 I took the most important decision from SLE-2 and SLE-3, choosing A* with Manhattan distance over BFS, and justified it with my profiling results and design.

PART C: AI Contribution Note

- **AI tools used (if any):** GitHub Copilot in SLE-1 and SLE-2, Claude in SLE-2 and SLE-3, Gemini in SLE-3, and Claude for this SLE-4.
- **What AI helped with:** Generating the basic chatbot logic and README layout in SLE-1, troubleshooting the py-spy command and structuring the SLE-2 and SLE-3 reports, and structuring this ADR and preparing viva questions.
- **What I did myself:** Reviewed and simplified the SLE-1 chatbot code, implemented BFS and A* for SLE-2, ran py-spy profiling and collected the metrics, designed the component boundaries and drew the C4 diagrams on draw.io for SLE-3, wrote the design rationale, and made and can defend the final decision.

PART D: Viva Preparation Notes

- A* uses f(n) = g(n) + h(n): g is the cost so far and h is the Manhattan distance estimate to the goal.
- My numbers: BFS expanded 33 nodes, A* 5 nodes; average time 0.044 ms vs 0.028 ms; same optimal solution. Timing differences are tiny, so node count is the stronger evidence.
- Admissible means the heuristic never overestimates the real remaining cost, which is why A* stays optimal. Manhattan distance qualifies because one move shifts one tile by one square.
- C4 Container view: Input Module → Search Engine → Output Module, with the Heuristic Calculator and Memory State (Visited Set) supporting the Search Engine.
- For bigger problems (15-puzzle), BFS becomes infeasible and A* hits memory limits, so I would use IDA* or a stronger heuristic.